



Seminar Chuyên Đề: Tìm Hiểu Công Nghệ Vibe Coding & GitHub Copilot

Trường Đại học Sài Gòn - Khoa Công nghệ Thông tin

Giảng viên hướng dẫn: TS. Đỗ Như Tài

TP. Hồ Chí Minh, tháng 02, năm 2026

Phân Công Công Việc Nhóm



**Thái Nguyễn Thanh
Danh**

03-04-Làm báo cáo



Đỗ Gia Huy
00-02-Làm báo cáo



Vũ Văn Tài
05-Làm báo cáo



Lưu Chấn Vinh
01-Làm slide



Chương 1: Giới Thiệu Đồ Án

Bối cảnh Đồ án

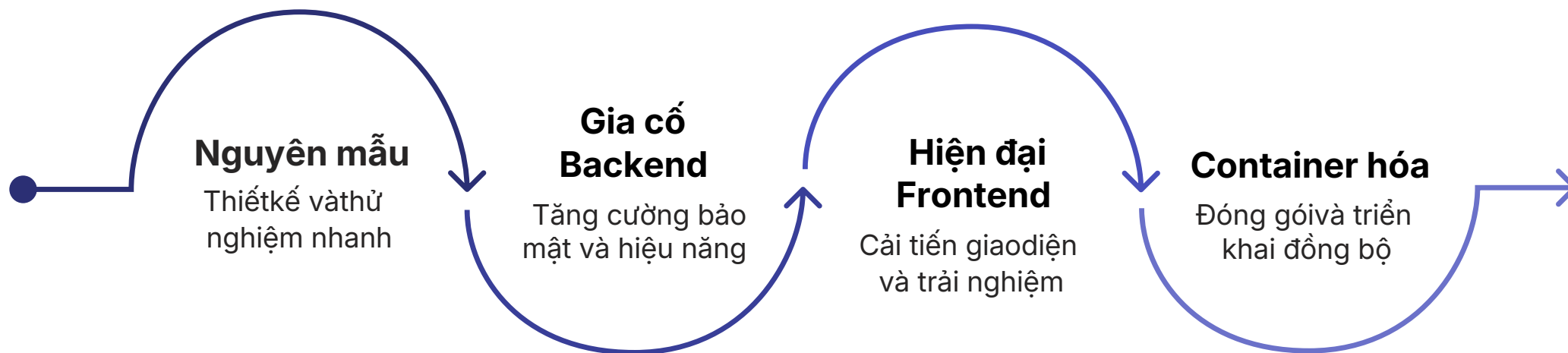
Ứng dụng AI vào lập trình (AI-assisted coding) là xu hướng tất yếu. Đồ án này nằm trong khuôn khổ GitHub Copilot Vibe Coding Workshop, nhằm làm chủ quy trình phát triển phần mềm hiện đại với GitHub Copilot Agent Mode và MCP.

Giới thiệu về Contoso

Contoso là công ty giả định chuyên kinh doanh sản phẩm ngoài trời. Họ cần một website mạng xã hội nhỏ để quảng bá sản phẩm và kết nối khách hàng.

Thách thức: Dự án thay đổi công nghệ (Python/JS sang Java/.NET) và yêu cầu triển khai linh hoạt (Containerization).

Chương 2: Tổng Quan Dự Án



Dự án mô phỏng môi trường thực tế khắc nghiệt, từ khởi đầu nhanh chóng đến chuyển đổi nền tảng và triển khai đồng bộ.

Phương Pháp Luận "Vibe Coding"

Vai trò Lập trình viên

Chuyển từ người viết mã sang Kiến trúc sư & Người ra quyết định.

Công cụ cốt lõi: GitHub Copilot Agent Mode + MCP.



Lập trình viên cung cấp ngữ cảnh và yêu cầu, AI thực thi chi tiết, sau đó lập trình viên kiểm tra và tinh chỉnh.

Chương 3: Thiết Kế Luận Lý

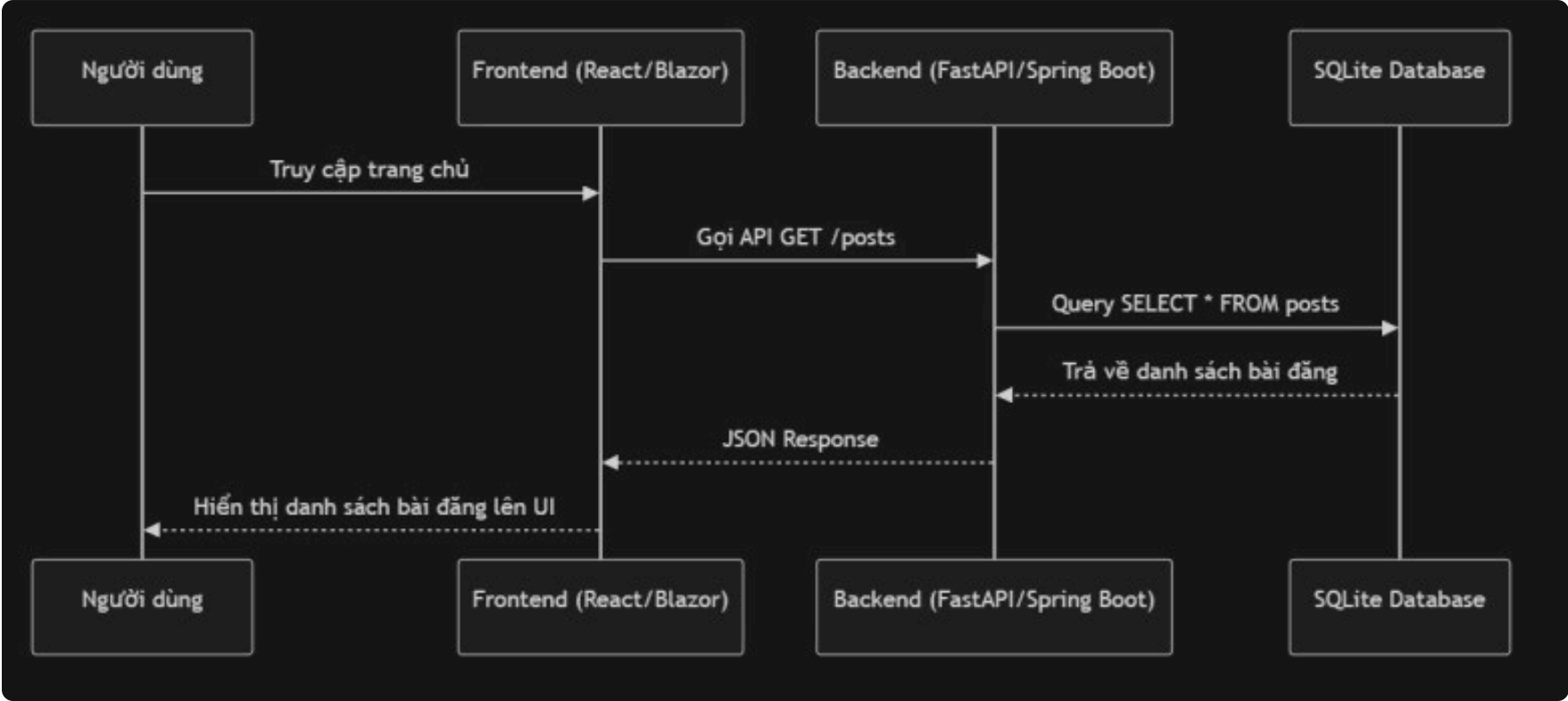
Chương này mô tả cách các thành phần giao tiếp và cách dữ liệu được tổ chức.

Kiến trúc Hướng API

Logic cốt lõi nằm ở bản thiết kế API. Định nghĩa tệp openapi.yaml trước tiên làm "Hợp đồng" bất biến giữa Frontend và Backend.

Mô hình Luồng Dữ liệu

Hệ thống hoạt động theo mô hình Client-Server cổ điển, tối ưu hóa cho container.



Hình 1: Sơ đồ luồng dữ liệu hệ thống

Giai đoạn 1: Prototyping

Chuyển ý tưởng thành sản phẩm hoạt động thực tế nhanh chóng.



Thiết lập "Hợp đồng" Giao tiếp

Sử dụng AI phân tích yêu cầu, sinh ra đặc tả OpenAPI (Swagger) làm "hợp đồng bất biến" giữa Frontend và Backend.



Backend Tốc độ cao (Python & FastAPI)

Python và FastAPI giúp sinh code chính xác, tự động tạo tài liệu API. Dùng SQLite để tập trung vào logic nghiệp vụ.



Frontend Trực quan (JavaScript & React)

React và Vite. Áp dụng Design-to-Code qua MCP: AI sinh mã React từ thiết kế Figma, giảm thiểu sai sót.

Giai đoạn 2: Backend Hardening (Củng cố Backend)

Chuyển đổi Backend từ Python (FastAPI) sang Java (Spring Boot) giữ nguyên tính năng và đặc tả API cũ.

Mục tiêu:

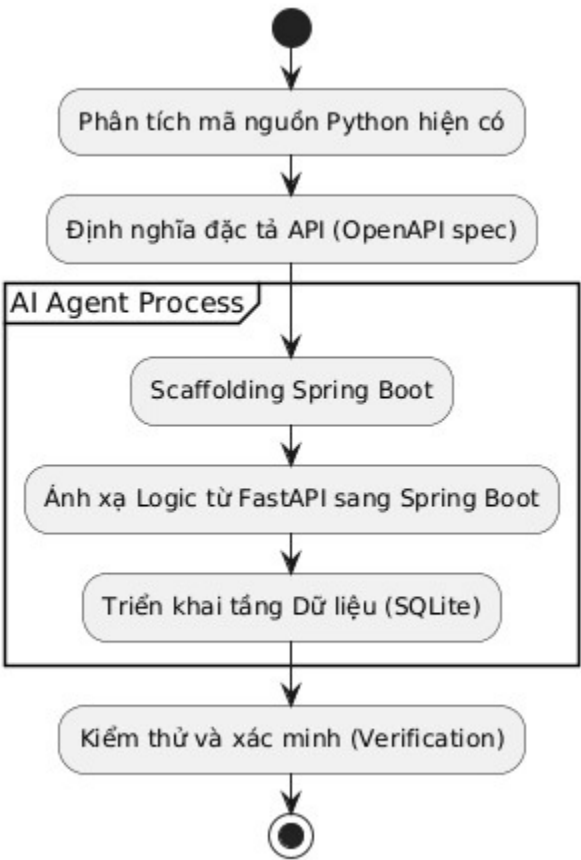
Chuyển đổi mượt mà các Endpoint mà không thay đổi trải nghiệm Front-end.

Đối tượng tác động:

Backend mới cần đáp ứng các yêu cầu nghiệp vụ từ phòng Marketing của Contoso.

Ràng buộc:

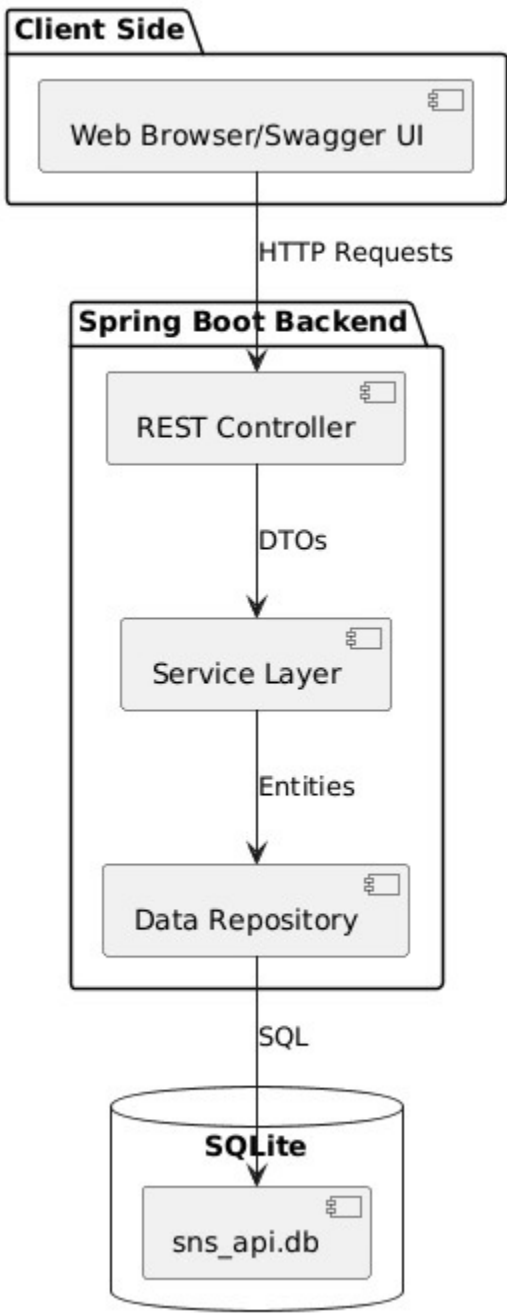
Giữ nguyên cấu trúc dữ liệu và đường dẫn API trong openapi.yaml.



Hình 2: Sơ đồ luồng hoạt động

Ảnh xạ Kiến trúc:

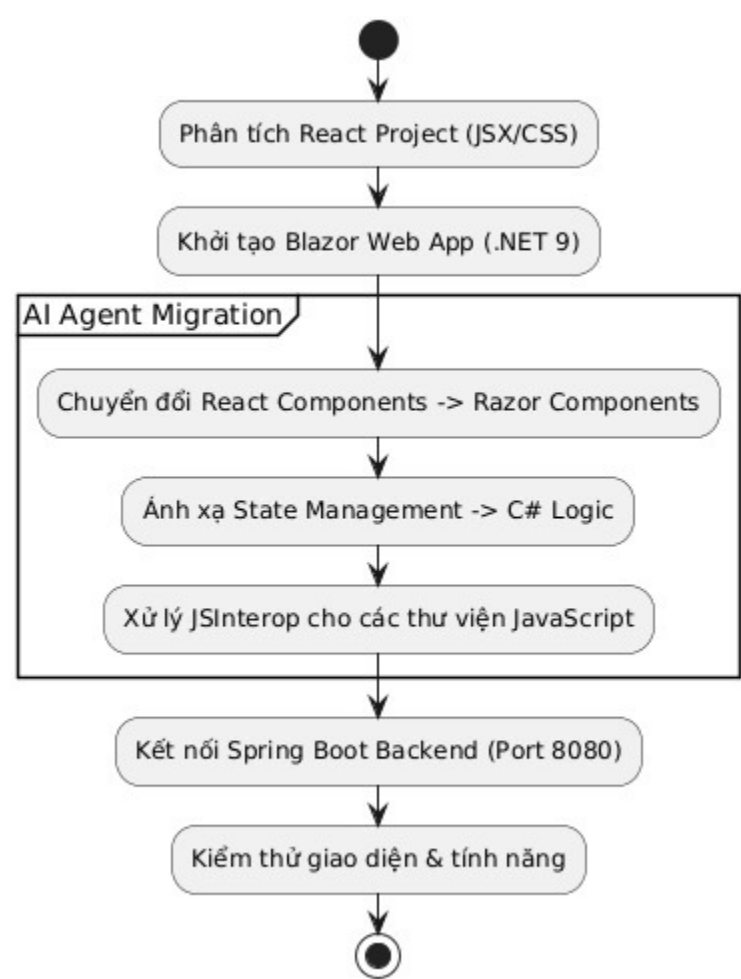
Hệ thống thiết kế theo mô hình Layered Architecture chuẩn của Spring Boot.



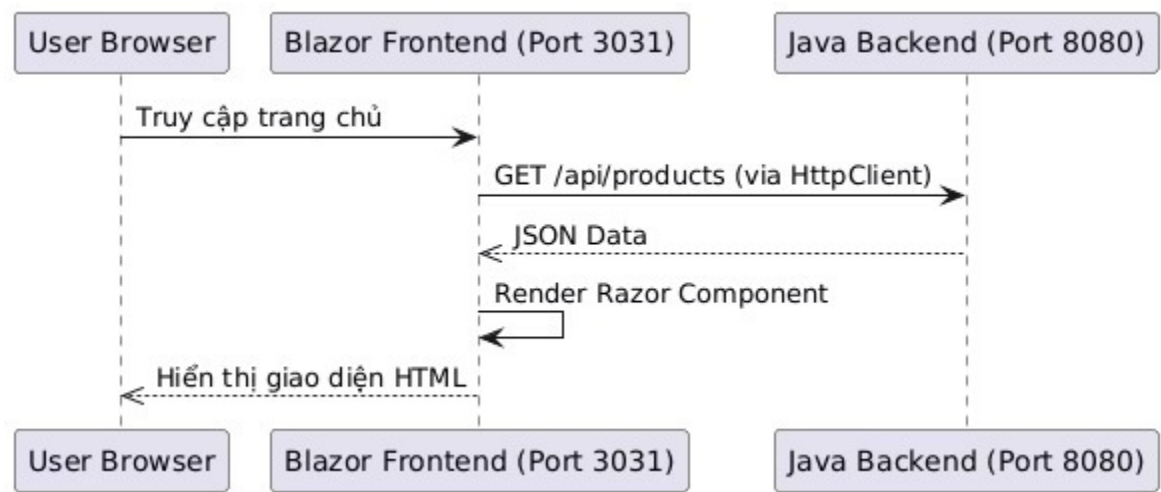
Hình 3: Sơ đồ các thành phần

Giai đoạn 3: Frontend Modernization

Chuyển đổi Frontend từ React (JavaScript) sang Blazor (.NET) để tận dụng hệ sinh thái .NET.



Hình 4: Sơ đồ quy trình migration



Hình 5: Lược đồ Sequence thể hiện tương tác hệ thống

Mục tiêu:

- Tận dụng hệ sinh thái .NET và tăng khả năng bảo mật, hiệu suất thông qua kiến trúc Single Page Application (SPA) của Microsoft.
- Frontend chuyển đổi phức tạp do phải đảm bảo tương thích giao diện (CSS) và xử lý sự kiện.
- Áp dụng phương pháp chuyển đổi theo từng component (Component-based Migration).

Trải nghiệm người dùng (UX):

- Giữ nguyên thiết kế UI/UX từ bản React cũ.

Kiến trúc hệ thống:

- Frontend (Blazor) sẽ đóng vai trò là Consumer, tiêu thụ các API từ Java Backend.

Phạm vi:

- Di chuyển toàn bộ các trang chính (Home, Product List, Social Feed, Profile).

Giai đoạn 4: Containerization

Mục tiêu: Chuyển đổi mô hình ứng dụng truyền thống sang kiến trúc Container-native để loại bỏ sự phụ thuộc vào môi trường máy chủ cục bộ.

Multi-stage Build

Tách biệt môi trường "Xây dựng" và "Thực thi" để tối ưu dung lượng Image và tăng cường bảo mật.

Service Discovery

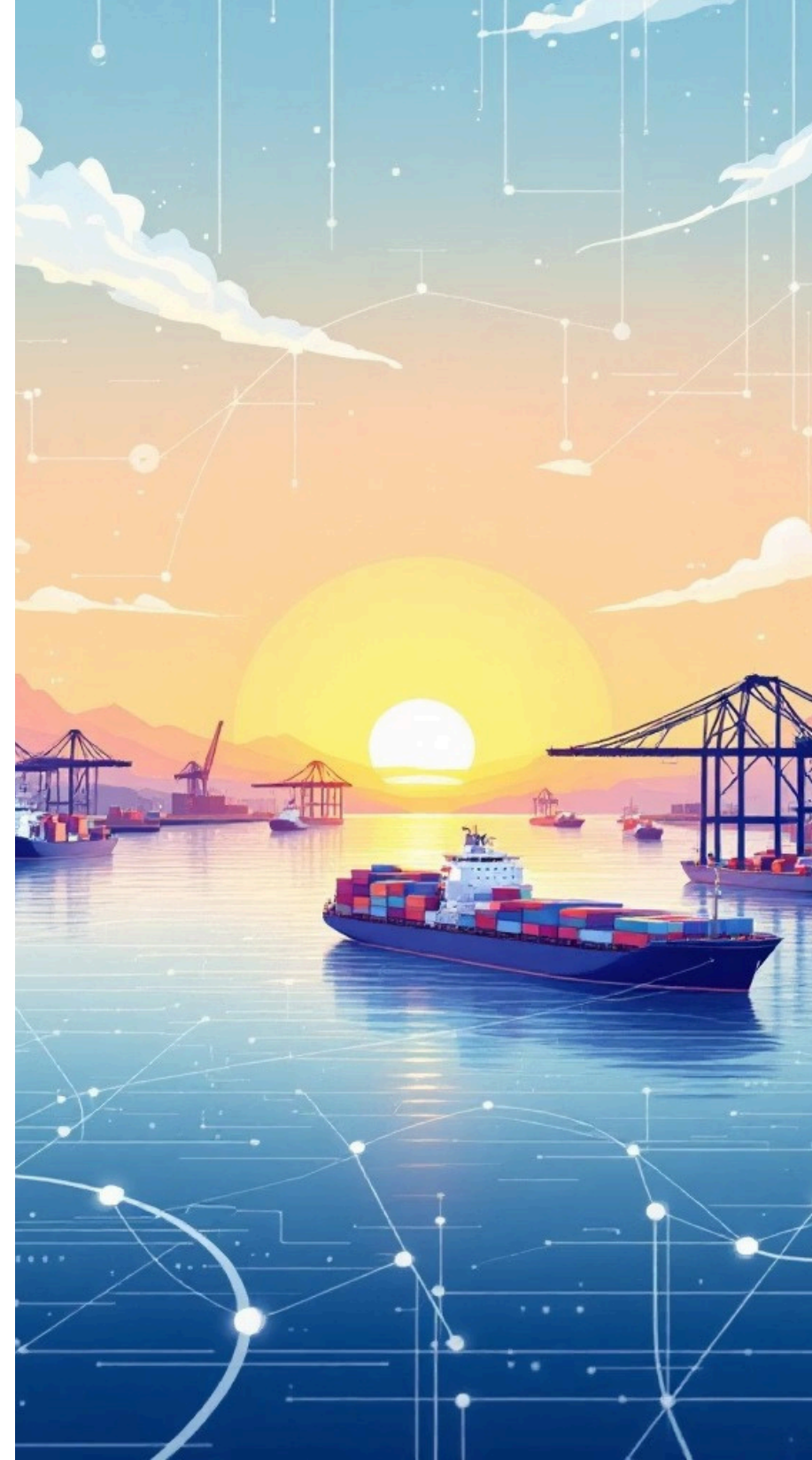
Thiết lập cơ chế định danh dịch vụ nội bộ (Frontend kết nối Backend qua tên dịch vụ).

Cấu hình Mạng

Khởi tạo mạng ảo biệt lập "contoso" để các container giao tiếp an toàn.

Quản lý Cấu hình

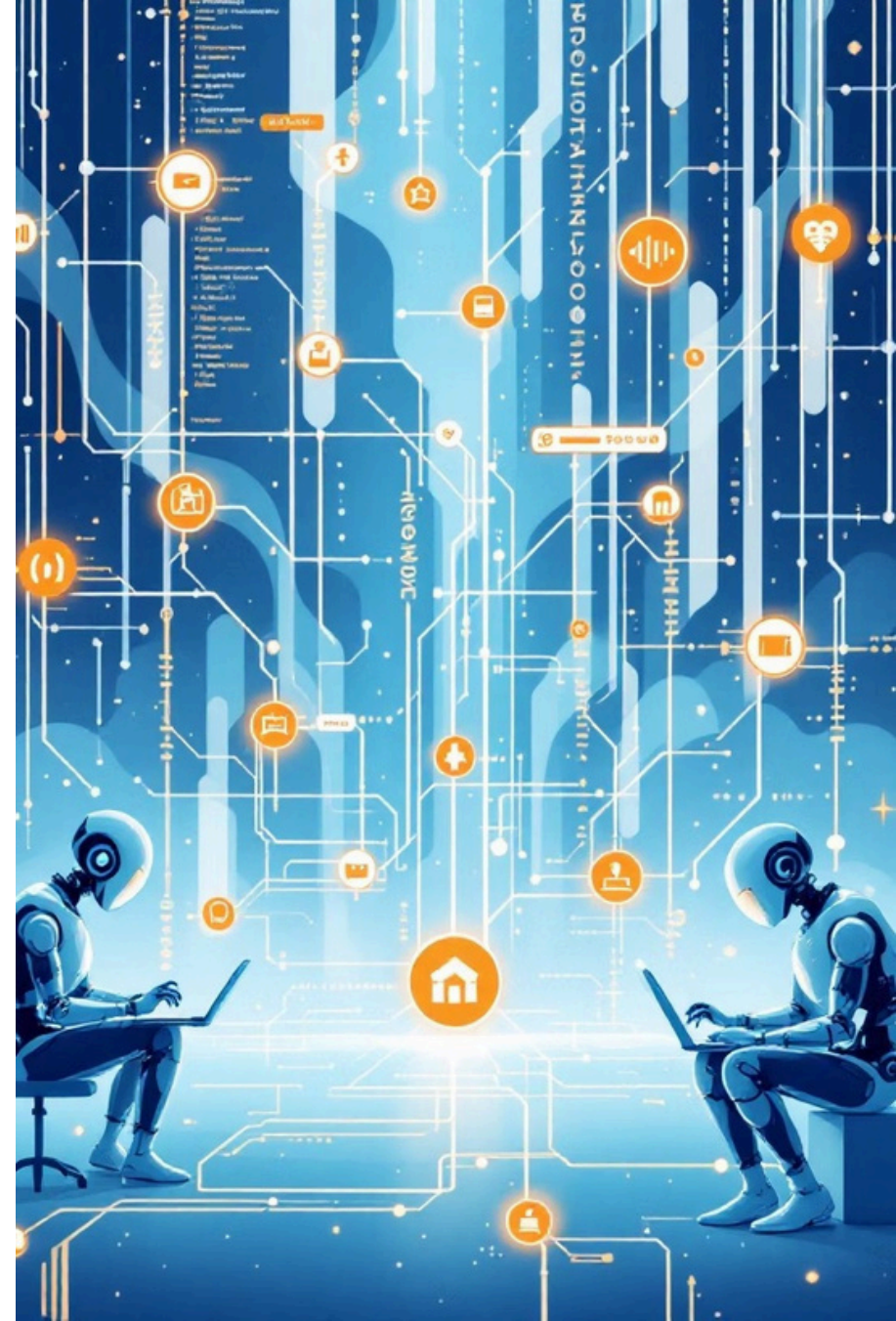
Sử dụng biến môi trường để truyền tham số kết nối vào container mà không sửa đổi mã nguồn.



CHƯƠNG 4: THỰC THI VẬT LÝ

Chương này trình bày chi tiết các bước triển khai kỹ thuật, từ thiết lập môi trường, cấu hình giao thức ngữ cảnh cho AI, đến việc xây dựng và chuyển đổi mã nguồn qua các ngôn ngữ khác nhau.

Quy trình này tuân thủ nghiêm ngặt phương pháp "Vibe Coding" – nơi lập trình viên điều phối AI Agent để thực thi tác vụ.



4.1. Thiết lập Môi trường và Cấu hình Ngữ cảnh



1. Chuẩn bị IDE và Extension

- Sử dụng Visual Studio Code(hoặc GitHub Codespaces).
- Cài đặt GitHub Copilot Extension (chế độ Agent Mode).
- Lựa chọn Model:Claude3.5Sonnet.

2. Cấu hình Model Context Protocol (MCP)

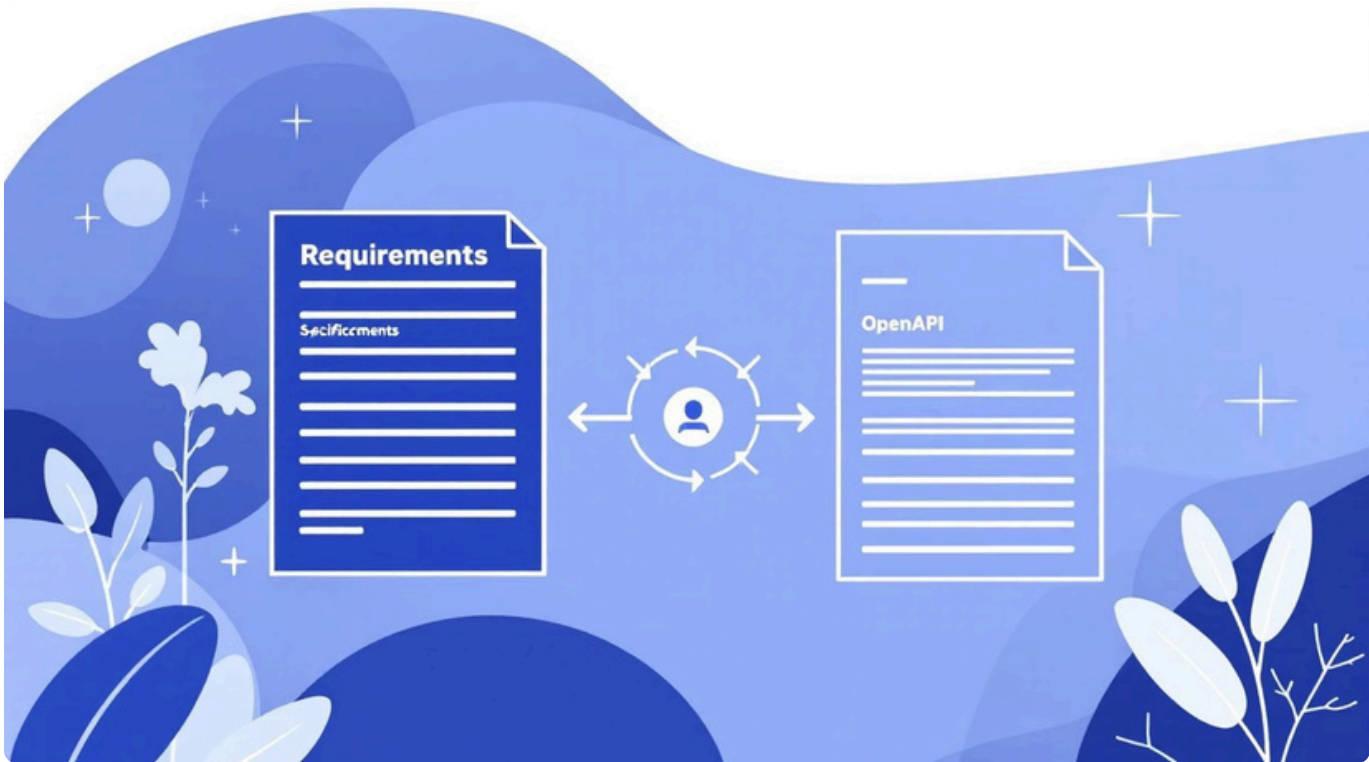
- Cài đặt context7server(đọc/ghi cấu trúc thư mục).
- Cài đặt Figma MCP server (truy cập file thiết kế Figma).
- Cài đặt Fetch server (truy cập internet).

3. Thiết lập "Luật chơi" (Custom Instructions)

- Sao chép filehướngdẫn (.github/copilot-instructions.md) vào thư mục gốc.
- Định hình phong cách code cho AI (ví dụ: "Luôn sử dụng Type Hints trong Python").

4. Khởi tạo Đặc tả API (OpenAPI Scaffolding)

- Input: product-requirements.md.
- Action: Copilot phân tích nghiệp vụ, sinh file openapi.yaml.
- Output: Thiết kế API chuẩn Swagger với các endpoints và cấu trúc dữ liệu JSON.



```

21 # FastAPI chỉ cần khởi tạo một object FastAPI() để khởi tạo (lưu ý in
22 # ra) và /comallone cresstro ()):
23 caderaticiv), (love cocter for "astapi" (ip), for setor )
24 low ehane toky "tle crows ent lon, thvel, (fest the rurt that is
25 # fatli is /ercallone pttel)
26 /# FastAPI, "( fast to come /" sevti) ("t (at (ly), b
27 centflic ratop" ((( ceemation: database ) APP)
28 /# python out action call);

```



4.2. Giai đoạn 1.1: Phát triển Backend với Python (FastAPI)

Mục tiêu: Xây dựng Backend nhanh chóng để kiểm chứng thiết kế API.



Thiết lập môi trường Python

Sử dụng `uv venv` để tạo môi trường ảo và kích hoạt.



Scaffold ứng dụng với AI

Prompt AI đọc `openapi.yaml`, sử dụng SQLite, cổng 8000, cấu hình CORS.



AI Action

Copilot tự động tạo `main.py`, `database.py`, `models.py`, `routers/`.



Kiểm thử và Vận hành

Chạy ứng dụng, truy cập Swagger UI để xác nhận API hoạt động đúng.



4.3. Giai đoạn 1.2: Phát triển Frontend với JavaScript (React)

Mục tiêu: Xây dựng giao diện người dùng dựa trên thiết kế Figma có sẵn.

01

Khởi tạo dự án React

Sử dụng Vite để tạo khung dự án, cài đặt dependencies, cấu hình cổng 3000.

03

AI Action

Copilot đọc dữ liệu vector, trích xuất mã màu, font-size, padding và sinh ra các Component React.

02

Hiện thực hóa thiết kế từ Figma

Dùng Figma MCP Server, prompt AI lấy thiết kế từ URL Figma, tạo trang Home với CSS Modules.

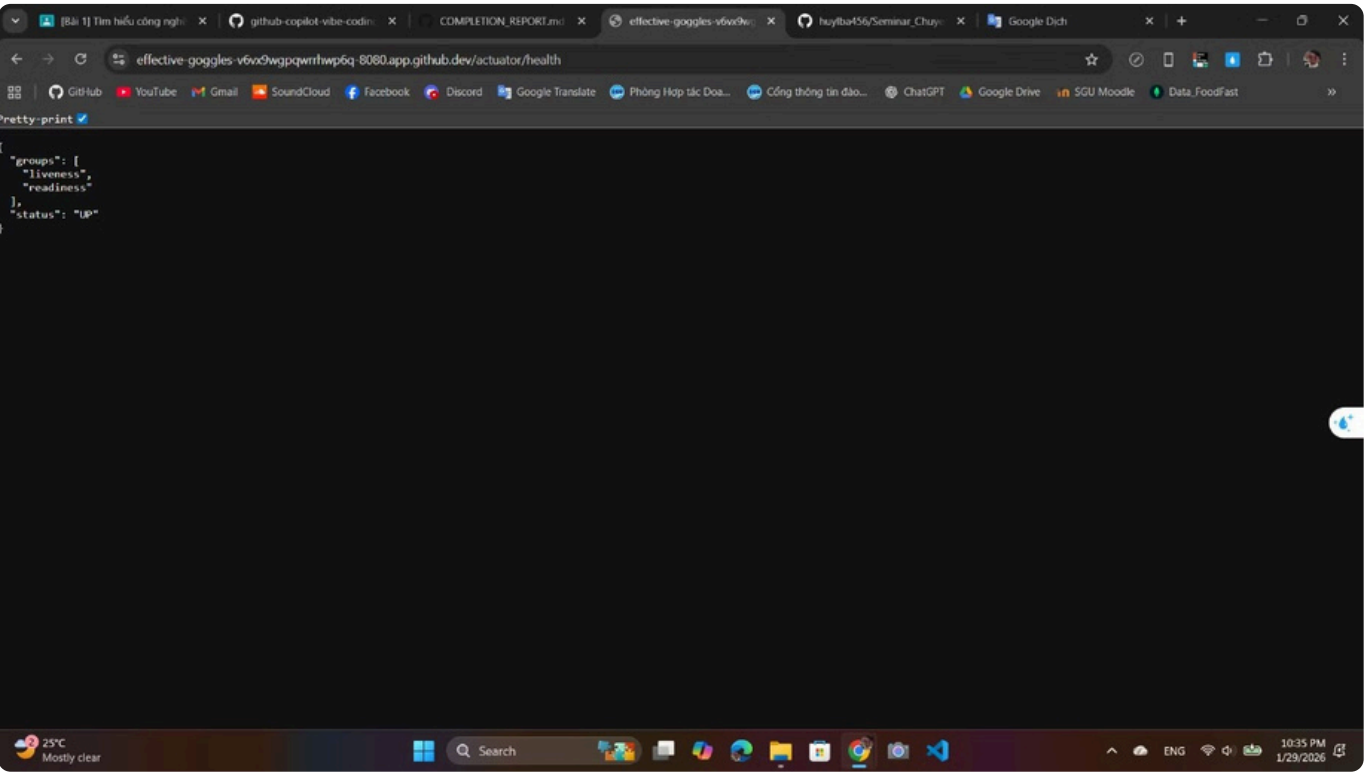
04

Tích hợp Logic

Prompt AI kết nối Frontend với Backend đang chạy tại <http://localhost:8000>, sinh code gọi API và render dữ liệu.

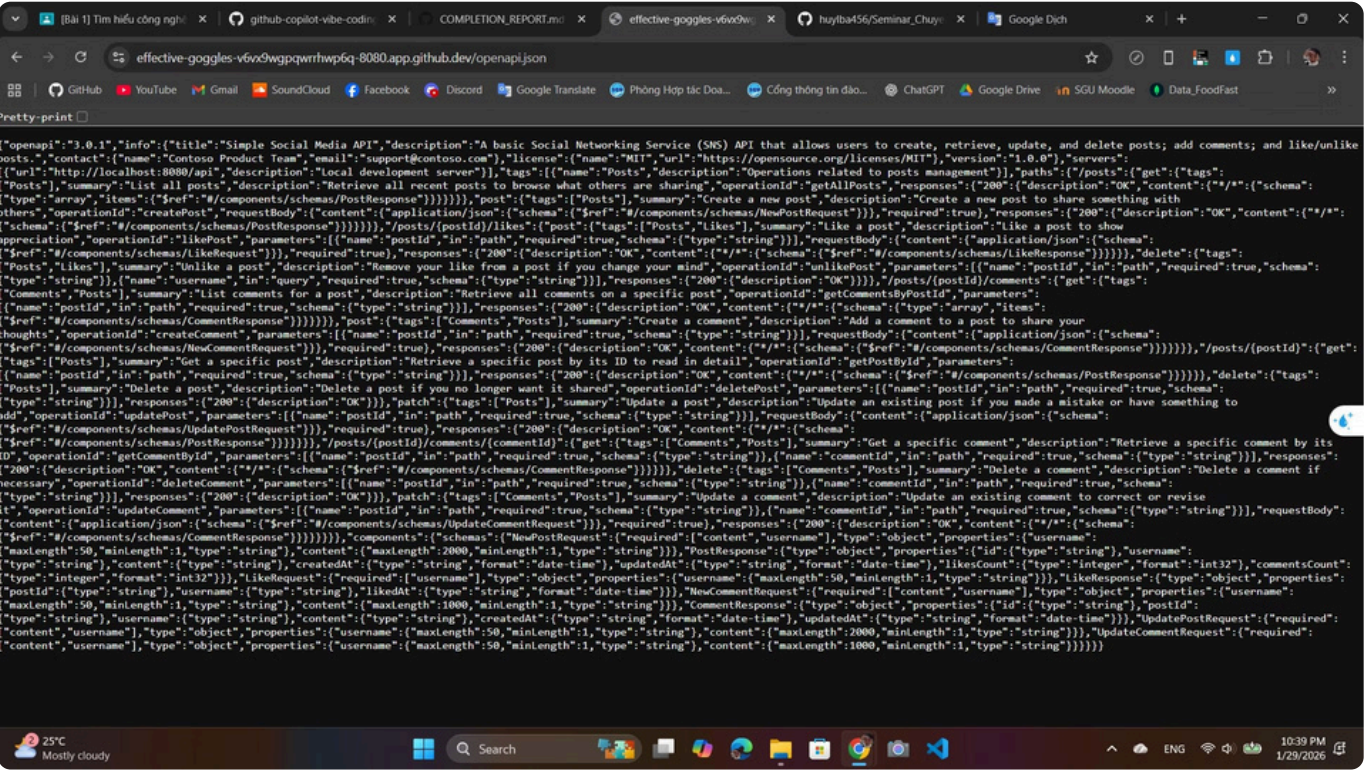
4.4. Giai đoạn 2: Chuyển đổi Backend sang Java (Migration)

Mục tiêu: Chuyển đổi nền tảng Backend sang Java Spring Boot mà không làm gián đoạn dịch vụ.



Khởi tạo dự án Spring Boot

Tạo cấu trúc Java với Gradle, thêm dependencies (Spring Web, JPA, Lombok, H2), cấu hình cổng 8080.



Thực hiện Migration thông minh

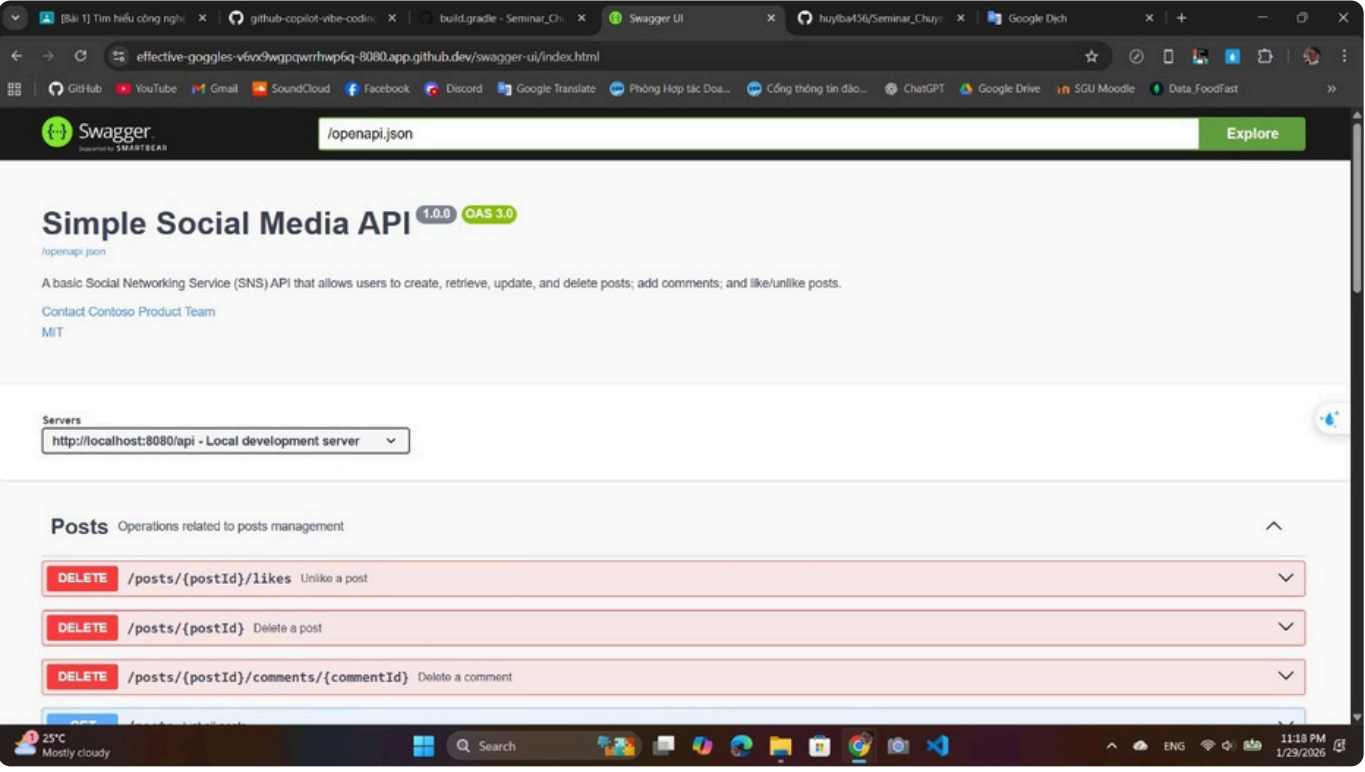
Prompt AI "dịch" code Python sang Java Spring Boot, đảm bảo giữ nguyên input/output theo openapi.yaml.

Chi tiết chuyển đổi

Python main.py -> Java Application.java, FastAPI Router -> Spring Controller, Pydantic Models -> Java Records/POJOs, SQLAlchemy ORM -> Spring Data JPA Repository.

Kiểm thử tương thích

Chạy ứng dụng Java, truy cập Swagger UI để so sánh kết quả với API Python cũ.



Hình 6, 7, 8: Các hình thể hiện migrate và chạy thành công Backend

4.5. Giai đoạn 3: Chuyển đổi Frontend sang .NET (Migration)

Mục tiêu: Chuyển đổi Frontend sang công nghệ Microsoft Blazor (C#).

Khởi tạo dự án Blazor

Sử dụng .NET CLI: `dotnetnewblazor`,
cấu hình cổng 3031 (http) và 3030 (https).

Chuyển đổi Component

Prompt AI chuyển đổi React component sang Blazor Razor Components (.razor).

Chi tiết chuyển đổi

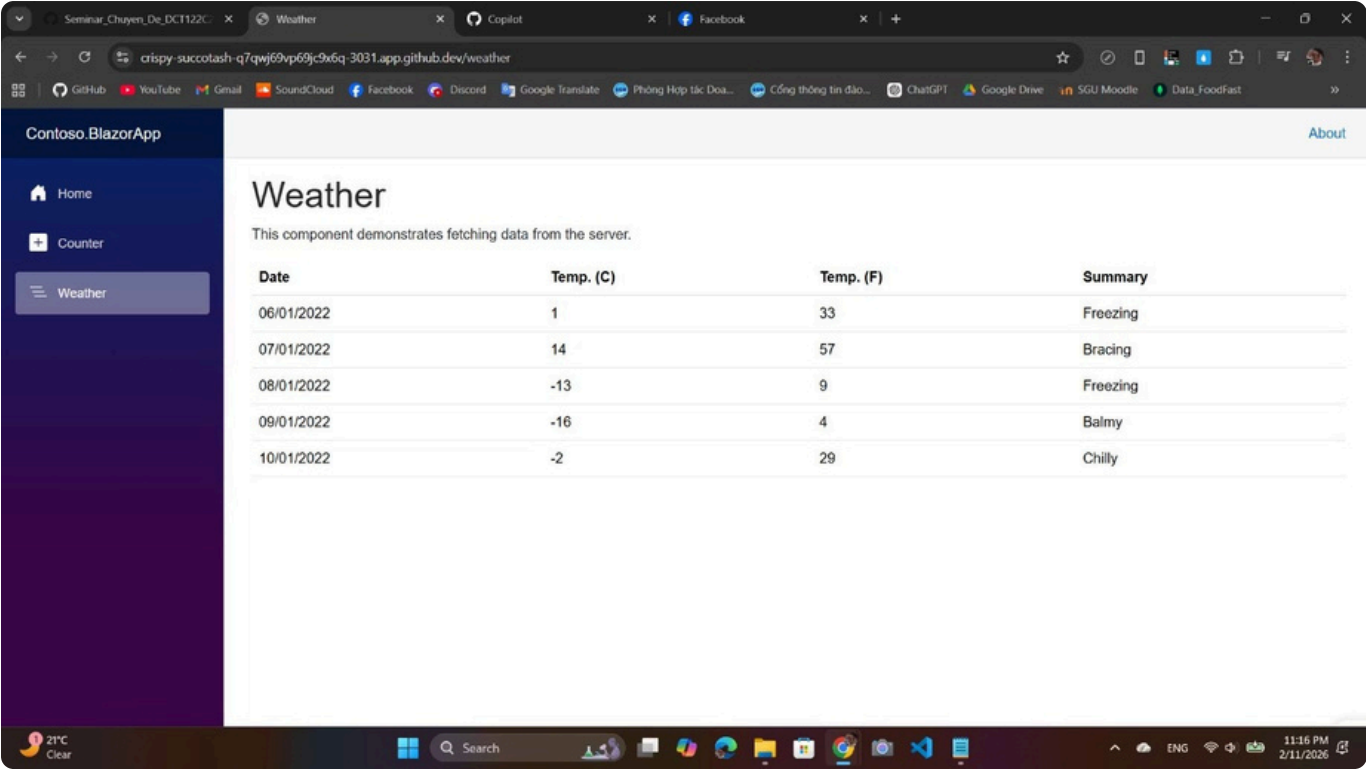
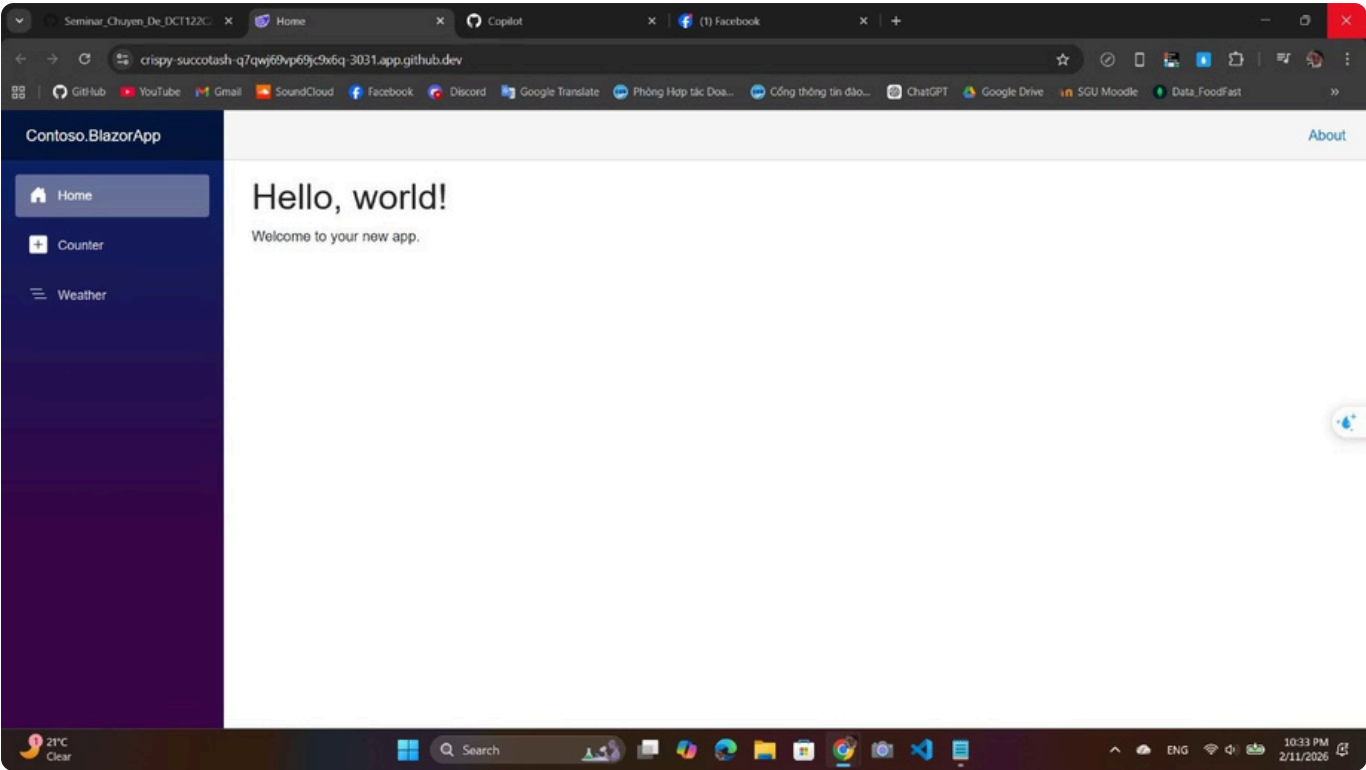
React JSX -> Razor HTML Syntax,
React Hooks -> Blazor Lifecycle Methods, JavaScript Logic -> C# Code Block.

Tích hợp API Backend

Đăng ký HttpClient, cấu hình Base Address trỏ về Backend Java (<http://localhost:8080>).

Kiểm thử tương thích

Build không lỗi, giao diện hiển thị chính xác, chức năng tương tác mượt mà.



Hình 9, 10: Hiển thị giao diện khi migrate xong

4.6. Giai đoạn 4: Đóng gói và Triển khai (Containerization)

Mục tiêu: Đóng gói toàn bộ ứng dụng (Java Backend + .NET Frontend) để chạy trên mọi môi trường.



Dockerfile cho Backend (Java)

Sử dụng Multi-stage Build `gradle:jdk21 -> eclipse-temurin:21-jre`, expose cổng 8080.



Dockerfile cho Frontend (.NET)

Multi-stage Build `dotnet/sdk:9.0 -> dotnet/aspnet:9.0`, expose cổng 8080.



Điều phối bằng Docker Compose

Tạo `compose.yaml` để chạy 2 container, cấu hình service, cổng, biến môi trường và mạng `contoso`.



Thực thi Final

Chạy `dockercompose up --build` Hệ thống khởi động, Frontend tại `localhost:3030` gọi Backend Java tại `localhost:8080`.

[+] Running 4/4

✓ seminar_chuyen_de_dct122c2-contoso-backend	Built	0.0s
✓ seminar_chuyen_de_dct122c2-contoso-frontend	Built	0.0s
✓ Container seminar_chuyen_de_dct122c2-contoso-backend-1	Running	0.0s
✓ Container seminar_chuyen_de_dct122c2-contoso-frontend-1	Started	0.3s

Hình 11: Hiện thị đã container hóa và build xong frontend và backend



**THANK
YOU**